CAMPUS RECRUITMENT TRAINING & PLACEMENT BOARD

OFFICIAL PRESCRIBED PLACEMENT MASTER TEXTBOOK

Core Java Programming & Placement Foundations (Prescribed Master Textbook)

Subject Track: Java

Edition: 2026 Comprehensive Campus Placement Edition

SYLLABUS & CURRICULUM OVERVIEW

Complete, authoritative campus placement master textbook covering all Java programming language topics: JVM Internals, Memory Model, OOP Architecture, Strings & Memory Pooling, Collections, Generics, Concurrency, Stream API & Lambdas, I/O, Modern Java (Java 9 to 21 LTS), and Placement Puzzles.

STRICT EXAMINATION GROUNDING GUARANTEE

All campus placement assessments, live code challenges, milestone tests, and company interview drills for Java on this platform are derived strictly and exclusively from the theoretical concepts, algorithms, worked examples, and placement problems documented in this official textbook. Mastery of these chapters guarantees 100% exam preparation.

PRESCRIBED CHAPTER SYLLABUS (10 CHAPTERS):

Ch 1: JVM Architecture, Memory Model & Bytecode Execution

Ch 2: Object-Oriented Programming Mastery

Ch 3: Java Memory Management, Garbage Collection & String Pool

Ch 4: Exception Handling & Robust Error Architecture

Ch 5: Java Collections Framework Deep Dive

Ch 6: Java Generics, Reflection & Annotations

Ch 7: Multithreading, Concurrency & Java Memory Model

Ch 8: Java 8+ Functional Programming & Streams API

Ch 9: I/O Streams, Serialization & Modern Networking

Ch 10: Java Placement Coding Patterns & Enterprise Architecture

Author: Prescribed Courseware Architectural Board

Verified Complete Unabridged Placement Master Edition | Internal Campus Circulation

Edition: 2026 Comprehensive Campus Placement Edition

Subject: Java

Courseware Board: Prescribed Placement Architectural Board

Status: In-App Complete Master Textbook (Full Curriculum Available In-Platform)

> **? EXAM GROUNDING POLICY:** All campus placement technical assessments, coding lab challenges, and technical interview evaluations are strictly grounded in the concepts, algorithms, formulas, step-by-step solved examples, practice questions, and interview questions detailed in this textbook.

· Course Overview & Curriculum Scope

Complete, authoritative campus placement master textbook covering all Java programming language topics: JVM Internals, Memory Model, OOP Architecture, Strings & Memory Pooling, Collections, Generics, Concurrency, Stream API & Lambdas, I/O, Modern Java, and Placement Puzzles.

· Complete Table of Contents (Chapter-by-Chapter)

- [Chapter 1: JVM Architecture, Memory Model & Bytecode Execution](#chapter-1)
- [Chapter 2: Object-Oriented Programming Mastery](#chapter-2)
- [Chapter 3: Java Memory Management, Garbage Collection & String Pool](#chapter-3)
- [Chapter 4: Exception Handling & Robust Error Architecture](#chapter-4)
- [Chapter 5: Java Collections Framework Deep Dive](#chapter-5)
- [Chapter 6: Java Generics, Reflection & Annotations](#chapter-6)
- [Chapter 7: Multithreading, Concurrency & Java Memory Model](#chapter-7)
- [Chapter 8: Java 8+ Functional Programming & Streams API](#chapter-8)
- [Chapter 9: I/O Streams, Serialization & Modern Networking](#chapter-9)
- [Chapter 10: Java Placement Coding Patterns & Enterprise Architecture](#chapter-10)

CHAPTER 1

JVM Architecture, Memory Model & Bytecode Execution

Key Principle / Exam Takeaway: Master JDK vs JRE vs JVM, classloader subsystems, runtime data areas (Heap, Stack, Metaspace), and JIT compiler optimization.

Core Curriculum Topics Covered in this Chapter:

- JVM Architecture: ClassLoader, Execution Engine, JIT Compiler, Native Interface
- Runtime Data Areas: Heap, Thread Stack, Program Counter (PC), Native Method Stack, Metaspace
- Bytecode verification, Just-In-Time (JIT) compilation, and HotSpot profiling
- Stack Frame structure: Local Variable Array, Operand Stack, Frame Data
- Primitive types memory footprint vs Object reference headers

· 1. Core Concepts & In-Depth Explanation

Java achieves platform independence ('Write Once, Run Anywhere') by compiling human-readable source code (`.java`) into intermediate bytecode (`.class`) rather than machine-specific binary. The Java Virtual Machine (JVM) executes this bytecode on host operating systems.

The JVM runtime memory is partitioned into distinct zones:

- **Heap Memory:** Shared across all threads. Stores all instantiated objects and instance arrays. Managed automatically by the Garbage Collector.
- **Thread Stack:** Dedicated per thread. Contains stack frames allocated for each method invocation. Stores primitive local variables and object reference addresses. Deallocated automatically upon method return.
- **Metaspace (Java 8+):** Stores class metadata, bytecode instructions, static variables, and runtime constant pools in native OS memory (replacing the legacy PermGen).
- **Execution Engine:** Combines an interpreter (for rapid initial startup) and the JIT (Just-In-Time) compiler (C1/C2 compilers that translate frequently executed 'hot' bytecode loops directly into native machine code).

· 2. Key Formulas, Rules & Complexity Classes

1. **Java Object Header Memory Overhead (64-bit JVM):** $$\text{Object Header} = \text{Mark Word (8 bytes)} + \text{Klass Pointer (4 bytes with compressed OOPs)} = 12 \text{ bytes}$$ $$\text{All objects padded to multiples of 8 bytes (8-byte alignment rule)}!$$ 2. **Stack vs Heap Lifetime:** $$\text{Stack: } LIFO \text{ lifecycle, strictly deterministic, zero GC overhead}$$ $$\text{Heap: } Dynamic lifetime, non-deterministic, managed via GC mark-and-sweep$$

· 3. Step-by-Step Solved Examples

Example 1.1: Understanding Stack Frame Allocation & Object References

- **Difficulty / Target Companies:** `Easy | Amazon, TCS, Infosys`
- **Problem Statement:** Trace the memory allocation of primitives and object references in the Heap vs Stack during method execution.

Step-by-Step Walkthrough & Logic:

- Step 1: When `main()` executes, a stack frame is pushed to the main thread's Stack.
- Step 2: Primitive variable `int x = 42` is allocated directly inside the stack frame's Local Variable Array.
- Step 3: `new StringBuilder("Placement")` allocates a dynamic object on the Heap. The variable `sb` on the stack stores a 4-byte reference pointer pointing to the heap address.
- Step 4: Passing `sb` to a method passes a copy of the reference pointer (Pass-By-Value). Modifications through the reference mutate the heap object.

Complete Solved Code / Calculation:

```
public class MemoryDemo {
    public static void main(String[] args) {
        int primitiveVal = 100; // Stored in Stack Frame
        StringBuilder refVal = new StringBuilder("Java"); // Object on Heap, pointer on Stack

        modify(primitiveVal, refVal);

        System.out.println("primitiveVal = " + primitiveVal); // Still 100
        System.out.println("refVal = " + refVal);             // "Java Placement"
    }

    public static void modify(int x, StringBuilder sb) {
        x = 999;                    // Mutates local stack copy only
        sb.append(" Placement");    // Mutates shared Heap object
    }
}
```

- **Line-by-Line Breakdown & Complexity Analysis:** Java is strictly pass-by-value. For primitives, the raw value is copied. For objects, the reference pointer is copied. Both references point to the same physical object in heap memory.

· 4. Practice Questions with Answers & Explanations

Practice Question 1.1

Question: Where are static variables stored in Java 8 and beyond?

Options:

- A) Java Thread Stack
- B) PermGen space
- C) Heap Memory (associated with the Class object in Metaspace)
- D) CPU Registers

Answer: **C) Heap Memory (associated with the Class object in Metaspace)** **Detailed Solution & Reasoning:** Starting in Java 8, PermGen was removed. Class metadata resides in native Metaspace, while static variables and interned strings reside on the regular Java Heap.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 1.1 (Amazon)

- **Interview Question:** _Is Java pass-by-reference or pass-by-value? Give a definitive technical explanation._

- **Model High-Scoring Answer:** Java is strictly pass-by-value, without exception. When an object is passed as an argument to a method, what is passed is the value of the reference (memory address pointer), copied into the method's local stack frame. If you reassign the reference inside the method (`obj = new Object()`), the caller's reference remains unchanged. If you invoke mutator methods (`obj.setValue(...)`), you mutate the shared heap object.

- **Interviewer Follow-Up & Pro Tip:** ? Emphasize reassigning the parameter reference to prove pass-by-value.

CHAPTER 2

Object-Oriented Programming Mastery

Key Principle / Exam Takeaway: Master the 4 pillars of OOP: Encapsulation, Inheritance, Polymorphism (static vs dynamic), and Abstraction (Abstract Classes vs Interfaces).

Core Curriculum Topics Covered in this Chapter:

- Encapsulation: Access modifiers (private, default, protected, public) and immutable class design
- Inheritance: IS-A vs HAS-A composition relationships and the super keyword
- Polymorphism: Method Overloading (compile-time) vs Method Overriding (runtime dynamic dispatch)
- Abstraction: Abstract Classes vs Interfaces (Default and static methods in Java 8+)
- SOLID Principles applied to enterprise Java design

· 1. Core Concepts & In-Depth Explanation

Object-Oriented Programming (OOP) models software systems as interacting objects that encapsulate state and behavior: 1. **Encapsulation:** Hiding internal object state behind accessor/mutator methods to maintain invariant integrity and prevent unauthorized coupling. 2. **Inheritance:** Code reuse mechanism where derived child classes inherit fields and methods from a base parent class (`extends`). Prefer Composition (`HAS-A`) over Inheritance (`IS-A`) to reduce tight coupling. 3. **Polymorphism:** Ability of an entity to take multiple forms. Compile-time polymorphism is achieved via Method Overloading (same method name, different parameter signatures). Runtime polymorphism is achieved via Method Overriding and Dynamic Method Dispatch via the virtual method table (vtable). 4. **Abstraction:** Exposing essential behavioral contracts while concealing concrete implementation details using Interfaces and Abstract Classes.

· 2. Key Formulas, Rules & Complexity Classes

1. **Dynamic Method Dispatch Resolution:** $$\text{Parent ref } = \text{new Child}(); \implies \text{ref.method}() \text{ resolves to Child's implementation at runtime via vtable}$$ 2. **Interface Evolution (Java 8+):** Interfaces can declare concrete `default` and `static` methods without breaking existing implementing classes.

· 3. Step-by-Step Solved Examples

Example 2.1: Creating an Immutable Class in Java

- **Difficulty / Target Companies:** `Medium | Goldman Sachs, Amazon, Morgan Stanley`
- **Problem Statement:** Design a fully thread-safe, immutable class containing a primitive and a mutable Date/List object.

Step-by-Step Walkthrough & Logic:

- Step 1: Declare the class `final` so it cannot be subclassed.
- Step 2: Make all fields `private` and `final`.
- Step 3: Do not provide setter methods.

- Step 4: Initialize all fields via constructor performing deep defensive copying of mutable arguments.

- Step 5: Return defensive copies in getter methods to prevent external mutation.

Complete Solved Code / Calculation:

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public final class ImmutableCandidate {
    private final int id;
    private final String name;
    private final List<String> skills; // Mutable list

    public ImmutableCandidate(int id, String name, List<String> skills) {
        this.id = id;
        this.name = name;
        // Defensive copy on construction
        this.skills = (skills != null) ? new ArrayList<>(skills) : new ArrayList<>();
    }

    public int getId() { return id; }
    public String getName() { return name; }

    // Return unmodifiable view or defensive copy
    public List<String> getSkills() {
        return Collections.unmodifiableList(skills);
    }
}
```

- **Line-by-Line Breakdown & Complexity Analysis:** Because `skills` is defensively copied on entry and wrapped in `unmodifiableList` on exit, callers cannot mutate the internal state of `ImmutableCandidate`.

· 4. Practice Questions with Answers & Explanations

Practice Question 2.1

Question: What happens if a subclass attempts to override a method marked `final` in the parent class?

Options:

- A) Code compiles but throws RuntimeException
- B) Compilation error
- C) Method is shadowed
- D) Method is executed only in debug mode

Answer: **B) Compilation error** **Detailed Solution & Reasoning:** The `final` keyword explicitly prevents method overriding at compile time.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 2.1 (Google)

- **Interview Question:** _Why should you prefer Composition over Inheritance in enterprise software design?_

- **Model High-Scoring Answer:** Inheritance breaks encapsulation because child classes depend intimately on implementation details of parent classes (fragile base class problem). Changes in parent methods can inadvertently break subclass invariants. Composition (`HAS-A`) provides loose coupling: components interact strictly through public interfaces, allowing runtime substitution of implementations (Strategy Pattern) and simplified unit testing via mocking.

- **Interviewer Follow-Up & Pro Tip:** ? Cite the Fragile Base Class problem and testability via dependency injection.

CHAPTER 3

Java Memory Management, Garbage Collection & String Pool

Key Principle / Exam Takeaway: Master JVM generations (Eden, Survivor, Tenured), Garbage Collection algorithms (G1, ZGC), and String Pool immutability.

Core Curriculum Topics Covered in this Chapter:

- Generational Garbage Collection hypothesis: Young Gen (Eden, S0, S1) vs Old Gen
- GC Algorithms: Serial, Parallel, CMS, G1 (Garbage-First), and low-latency ZGC
- String Pool (interning), String vs StringBuilder vs StringBuffer
- Memory leaks in Java: Unclosed resources, static collections, and thread locals
- Weak, Soft, and Phantom references

· 1. Core Concepts & In-Depth Explanation

Automatic memory management in Java relies on the **Weak Generational Hypothesis**: the empirical observation that most allocated objects have extremely short lifetimes and die shortly after creation.

The JVM Heap is split into generations: 1. **Young Generation:**

- **Eden Space:** All newly instantiated objects arrive here.
- **Survivor Spaces (S0 / S1):** Objects surviving a Minor GC are copied between Survivor spaces, incrementing an age counter.

2. **Old (Tenured) Generation:** Objects that survive a threshold number of Minor GC cycles (tenuring threshold, default 15) are promoted to the Old Generation. Major/Full GC sweeps the Old Generation.

String Constant Pool: Strings are immutable in Java. Literals (`String s = "test"`) reside in the String Pool inside Heap memory, allowing multiple references to share the identical physical character array. Using `new String("test")` forces the creation of a separate new object outside the pool.

· 2. Key Formulas, Rules & Complexity Classes

1. **String Comparison Invariant:** $$"java" == "java" \implies \text{TRUE (identical String Pool reference)}$$ $$"java" == \text{new String}("java") \implies \text{FALSE (different heap addresses)}$$ $$\text{Always use: } s1.\text{equals}(s2) \text{ for logical value equality!}$$

· 3. Step-by-Step Solved Examples

Example 3.1: Diagnosing String Pool Allocation & intern() Method

- **Difficulty / Target Companies:** `Easy | Infosys, Wipro, TCS`
- **Problem Statement:** Evaluate the output of string reference comparisons with and without intern().

Step-by-Step Walkthrough & Logic:

- Step 1: `s1 = "Hello"` places literal in String Pool.
- Step 2: `s2 = new String("Hello")` allocates a new object in general Heap.

- Step 3: `s3 = s2.intern()` looks up the pool; finds existing literal and returns pool address.

- Step 4: `s1 == s2` is false (distinct addresses). `s1 == s3` is true.

Complete Solved Code / Calculation:

```
public class StringPoolDemo {
    public static void main(String[] args) {
        String s1 = "Hello";
        String s2 = new String("Hello");
        String s3 = s2.intern();

        System.out.println(s1 == s2);      // false (Pool vs General Heap)
        System.out.println(s1 == s3);      // true  (Both point to Pool object)
        System.out.println(s1.equals(s2)); // true  (Identical character sequence)
    }
}
```

- **Line-by-Line Breakdown & Complexity Analysis:** The `intern()` method ensures that all identical strings return the canonical pooled reference, saving memory.

· 4. Practice Questions with Answers & Explanations

Practice Question 3.1

Question: Which Garbage Collector is designed for multi-gigabyte heaps to maintain pause times under 10 milliseconds?

Options:

- A) Serial GC
- B) Parallel GC
- C) ZGC (Z Garbage Collector)
- D) Mark-Sweep GC

Answer: **C) ZGC (Z Garbage Collector)** **Detailed Solution & Reasoning:** ZGC performs all heavy GC work concurrently with running application threads using colored pointers and load barriers.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 3.1 (Amazon)

- **Interview Question:** _Can a memory leak occur in Java despite automatic Garbage Collection? Give a realistic example._

- **Model High-Scoring Answer:** Yes. A memory leak in Java occurs when an object is no longer needed by application logic, but an active, reachable reference from a GC root prevents the Garbage Collector from reclaiming it. Common examples: 1) Static collections that continually append objects without eviction. 2) Unregistered event listeners or callbacks. 3) Unclosed system resources (database connections, file streams). 4) Un-cleared ThreadLocal variables in pooled thread environments.

- **Interviewer Follow-Up & Pro Tip:** ? Cite ThreadLocal leaks in web container thread pools (Tomcat).

CHAPTER 4

Exception Handling & Robust Error Architecture

Key Principle / Exam Takeaway: Master Checked vs Unchecked exceptions, try-catch-finally control flow, try-with-resources (AutoCloseable), and custom exception architecture.

Core Curriculum Topics Covered in this Chapter:

- Throwable hierarchy: Error vs Exception (Checked vs Unchecked/RuntimeException)
- try-catch-finally execution semantics and System.exit() behavior
- try-with-resources and AutoCloseable interface for leak prevention
- Custom domain exceptions and exception chaining (getCause())
- Exception anti-patterns: Swallowing exceptions and catch-all Exception blocks

· 1. Core Concepts & In-Depth Explanation

Java uses exceptions to separate error-handling code from normal business logic. All errors derive from `java.lang.Throwable`:

- **Error:** Serious hardware or JVM problems (e.g., `OutOfMemoryError`, `StackOverflowError`) that applications should not attempt to catch.
- **Exception (Checked):** Subclasses of `Exception` excluding `RuntimeException`. Represents recoverable conditions (e.g., `IOException`, `SQLException`) enforced at compile time.
- **RuntimeException (Unchecked):** Programming bugs or logic errors (e.g., `NullPointerException`, `IndexOutOfBoundsException`, `IllegalArgumentException`).

`try-with-resources` automatically closes any object implementing `java.lang.AutoCloseable` at block termination, even when exceptions are thrown, eliminating `finally` cleanup boilerplate.

· 2. Key Formulas, Rules & Complexity Classes

1. **Throwable Hierarchy:** $$\text{Throwable} \implies \begin{cases} \text{Error (Unchecked, JVM level)} \\ \text{Exception} \implies \begin{cases} \text{RuntimeException (Unchecked)} \\ \text{Checked Exceptions (Compile-time verified)} \end{cases} \end{cases}$$

· 3. Step-by-Step Solved Examples

Example 4.1: Automated Resource Management with try-with-resources

- **Difficulty / Target Companies:** `Easy | Amazon, TCS, Cognizant`
- **Problem Statement:** Safely read the first line of a file using try-with-resources, guaranteeing resource deallocation.

Step-by-Step Walkthrough & Logic:

- Step 1: Declare the `BufferedReader` inside the `try (...)` parentheses.
- Step 2: Read line.
- Step 3: The reader is automatically closed at the closing brace.

Complete Solved Code / Calculation:

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class ResourceSafety {
    public static String readFirstLine(String path) throws IOException {
        try (BufferedReader br = new BufferedReader(new FileReader(path))) {
            return br.readLine();
        } // br.close() is guaranteed to execute automatically here
    }
}
```

- **Line-by-Line Breakdown & Complexity Analysis:** Replaces bulky try-catch-finally patterns and handles suppressed exceptions automatically if both block and close() throw.

· 4. Practice Questions with Answers & Explanations

Practice Question 4.1

Question: When does a finally block NOT execute?

Options:

- A) When an unhandled RuntimeException is thrown
- B) When a return statement is encountered in the try block
- C) When System.exit(0) is called in the try block
- D) When an OutOfMemoryError occurs

Answer: **C) When System.exit(0) is called in the try block** **Detailed Solution & Reasoning:** System.exit() terminates the entire JVM process immediately, bypassing pending finally blocks.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 4.1 (Microsoft)

- **Interview Question:** _What is the difference between throw and throws in Java?_

- **Model High-Scoring Answer:** The `throw` keyword is used inside method bodies to explicitly instantiate and raise an exception object (`throw new IllegalArgumentException(...)`). The `throws` keyword is used in method signatures to declare to callers that this method may propagate checked exceptions, forcing the caller to either handle them with try-catch or declare them in its own throws clause.

- **Interviewer Follow-Up & Pro Tip:** ? Be crisp: throw acts on an instance, throws acts on method declaration.

CHAPTER 5

Java Collections Framework Deep Dive

Key Principle / Exam Takeaway: Master ArrayList vs LinkedList, HashMap internal collision resolution (hash buckets to Red-Black trees), ConcurrentHashMap, and Comparable vs Comparator.

Core Curriculum Topics Covered in this Chapter:

- Collection hierarchy: List, Set, Queue, Deque, Map
- ArrayList amortized resizing vs LinkedList node overhead
- HashMap internals: Hashing, bucket array, load factor (0.75), and treeification (Red-Black trees at threshold 8)
- ConcurrentHashMap: CAS (Compare-And-Swap) and synchronized bucket locks
- Comparable (natural order) vs Comparator (custom strategies)

· 1. Core Concepts & In-Depth Explanation

The Java Collections Framework (JCF) provides unified data structures:

- **List:** Ordered collections allowing duplicates. `ArrayList` uses contiguous arrays for $O(1)$ index lookup. `LinkedList` uses doubly-linked nodes with higher memory overhead.
- **Set:** Unique elements. `HashSet` uses an internal `HashMap` where values are dummy sentinels. `TreeSet` uses Red-Black tree for $O(\log n)$ sorted order.
- **Map:** Key-value pairs. `HashMap` computes bucket index via `(n - 1) & hash(key)`. When multiple keys map to the same bucket, elements chain as linked lists. If bucket length exceeds 8 and total map capacity ≥ 64, the bucket treeifies into a Red-Black tree, improving worst-case search from $O(n)$ to $O(\log n)$.

· 2. Key Formulas, Rules & Complexity Classes

1. **HashMap Bucket Index Formula:** $$\text{index} = (\text{capacity} - 1) \ \&\ \text{hash}(key)$$ 2. **Threshold for Resizing:** $$\text{Threshold} = \text{Capacity} \times \text{Load Factor (default 0.75)}$$

· 3. Step-by-Step Solved Examples

Example 5.1: Sorting Objects with Multiple Comparator Criteria

- **Difficulty / Target Companies:** `Easy | Amazon, TCS, Infosys`
- **Problem Statement:** Sort candidates primarily by score descending, and secondarily by name ascending.

Step-by-Step Walkthrough & Logic:

- Step 1: Use `Comparator.comparingInt(...).reversed()` for score descending.
- Step 2: Chain `.thenComparing(...)` for name alphabetical ascending.
- Step 3: Invoke `candidates.sort(...)`.

Complete Solved Code / Calculation:

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

class Candidate {
    String name;
    int score;
    Candidate(String n, int s) { name = n; score = s; }
}

public class SortDemo {
    public static void main(String[] args) {
        List<Candidate> list = new ArrayList<>();
        list.add(new Candidate("Alice", 85));
        list.add(new Candidate("Bob", 92));
        list.add(new Candidate("Charlie", 85));

        list.sort(Comparator.comparingInt((Candidate c) -> c.score)
                            .reversed()
                            .thenComparing(c -> c.name));
    }
}
```

- **Line-by-Line Breakdown & Complexity Analysis:** Chaining comparators creates concise, readable sorting strategies without implementing legacy verbose comparator classes.

· 4. Practice Questions with Answers & Explanations

Practice Question 5.1

Question: What is the contract between equals() and hashCode() in Java?

Options:

- A) If hashCode() is equal, equals() must return true
- B) If equals() returns true, hashCode() must return identical values for both objects
- C) They operate independently
- D) hashCode() is only used in lists

Answer: **B) If equals() returns true, hashCode() must return identical values for both objects** **Detailed Solution & Reasoning:** If two objects are logically equal via equals(), they must produce identical hash codes so hash-based collections place them in the same bucket.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 5.1 (Amazon)

- **Interview Question:** _How does ConcurrentHashMap achieve thread safety without locking the entire map like Hashtable?_
- **Model High-Scoring Answer:** Legacy Hashtable uses synchronized methods that lock the entire table for every read and write. ConcurrentHashMap uses fine-grained locking: reads are completely lock-free via volatile memory reads. Writes use Lock-Free Compare-And-Swap (CAS) to insert nodes into empty buckets, and lock only the single head node of the target bucket (`synchronized (node)`) when chaining into collisions. This allows concurrent writes across different buckets.

- **Interviewer Follow-Up & Pro Tip:** ? Contrast segment locking (Java 7) with CAS + bucket node synchronization (Java 8+).

CHAPTER 6

Java Generics, Reflection & Annotations

Key Principle / Exam Takeaway: Master type safety, type erasure, PECS rule (Producer Extends, Consumer Super), Reflection API, and custom runtime annotations.

Core Curriculum Topics Covered in this Chapter:

- Generics type safety and compile-time verification
- Type Erasure: How bytecode preserves backwards compatibility
- Wildcards: Upper bounded (? extends T) vs Lower bounded (? super T)
- PECS Rule: Producer Extends, Consumer Super
- Reflection API and custom annotations with RetentionPolicy.RUNTIME

· 1. Core Concepts & In-Depth Explanation

Generics provide compile-time type safety, eliminating runtime `ClassCastException` hazards.

Type Erasure: To ensure backwards compatibility with pre-generic Java versions, the Java compiler enforces generic type checks at compile-time and then erases all type parameters, substituting raw types or boundary types in generated bytecode.

PECS Rule:

- **Producer Extends (`? extends T`):** Use when you only READ elements from the collection.
- **Consumer Super (`? super T`):** Use when you WRITE elements into the collection.

· 2. Key Formulas, Rules & Complexity Classes

1. **PECS Mnemonic:** $$\text{Producer Extends (READ ONLY)} \iff \text{List<? extends Number> list}$$ $$\text{Consumer Super (WRITE ALLOWED)} \iff \text{List<? super Integer> list}$$

· 3. Step-by-Step Solved Examples

Example 6.1: Applying PECS to Generic Collections Copy

- **Difficulty / Target Companies:** `Medium | Google, Oracle`
- **Problem Statement:** Write a generic copy method that transfers numbers from a source list to a destination list.

Step-by-Step Walkthrough & Logic:

- Step 1: Source produces data: `List<? extends T> src`.
- Step 2: Destination consumes data: `List<? super T> dest`.
- Step 3: Iterate and add.

Complete Solved Code / Calculation:

```
import java.util.List;

public class GenericsHelper {
    public static <T> void copy(List<? super T> dest, List<? extends T> src) {
        for (T item : src) {
            dest.add(item);
        }
    }
}
```

- **Line-by-Line Breakdown & Complexity Analysis:** Guarantees complete compile-time type safety allowing flexible inheritance hierarchies.

· 4. Practice Questions with Answers & Explanations

Practice Question 6.1

Question: Can you instantiate a generic array directly such as `T[] arr = new T[10];`?

Options:

- A) Yes
- B) No, because of Type Erasure the exact component type is not known at runtime
- C) Only for Integer types
- D) Yes, if marked public

Answer: **B) No, because of Type Erasure the exact component type is not known at runtime** **Detailed Solution & Reasoning:** Arrays require reified runtime type information, which conflicts with generic type erasure.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 6.1 (Google)

- **Interview Question:** _Explain Type Erasure and its consequences in Java._
- **Model High-Scoring Answer:** Type Erasure replaces all generic type parameters with their bounds (or `Object` if unbounded) and inserts explicit casts where necessary in bytecode. Consequences: 1) You cannot use `instanceof` with generic types (`list instanceof List<String>` is illegal). 2) You cannot create instances of type parameters (`new T()`). 3) You cannot create generic arrays (`new T[10]`).
- **Interviewer Follow-Up & Pro Tip:** ? Cite backwards compatibility with Java 1.4 as the engineering rationale.

CHAPTER 7

Multithreading, Concurrency & Java Memory Model

Key Principle / Exam Takeaway: Master thread lifecycle, synchronized, volatile visibility, Deadlocks, ExecutorService thread pools, and ReentrantLock.

Core Curriculum Topics Covered in this Chapter:

- Thread lifecycle: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED
- Java Memory Model: Main Memory, CPU Caches, and the volatile keyword
- Synchronization: Intrinsic locks (monitors), synchronized methods, and reentrancy
- Deadlock conditions and prevention strategies
- ExecutorService, ThreadPoolExecutor, Callable, and Future

· 1. Core Concepts & In-Depth Explanation

Multithreading executes concurrent paths of execution within the same process, sharing Heap memory.

Java Memory Model (JMM) & `volatile`: CPUs retain local L1/L2 caches. If Thread A modifies a shared variable, the new value may reside in CPU cache without writing to Main Memory, leaving Thread B with a stale copy. Marking a variable `volatile` guarantees **Visibility** (all reads and writes bypass local caches directly to Main Memory) and establishes a **Happens-Before** relationship, preventing instruction reordering. It does NOT guarantee atomicity for compound operations like `count++`.

Thread Pools: Managing raw threads causes high overhead. `ExecutorService` pools reusable worker threads via blocking task queues (`LinkedBlockingQueue`).

· 2. Key Formulas, Rules & Complexity Classes

1. **Deadlock Conditions (Coffman Conditions):** $$\text{Mutual Exclusion} \land \text{Hold and Wait} \land \text{No Preemption} \land \text{Circular Wait}$$ 2. **Volatile vs Synchronized:** $$\text{volatile} = \text{Visibility only (Zero lock overhead)}$$ $$\text{synchronized} = \text{Visibility} + \text{Mutual Exclusion Atomicity}$$

· 3. Step-by-Step Solved Examples

Example 7.1: Thread-Safe Singleton with Double-Checked Locking

- **Difficulty / Target Companies:** `Medium | Amazon, Microsoft, Uber`
- **Problem Statement:** Implement a thread-safe, lazy-initialized Singleton in Java.

Step-by-Step Walkthrough & Logic:

- Step 1: Declare constructor private.
- Step 2: Declare static instance as `private static volatile`.
- Step 3: Check null twice: once without lock, then inside synchronized block.

Complete Solved Code / Calculation:

```
public class ThreadSafeSingleton {
    private static volatile ThreadSafeSingleton instance;

    private ThreadSafeSingleton() {}

    public static ThreadSafeSingleton getInstance() {
        if (instance == null) { // 1st check (no locking overhead)
            synchronized (ThreadSafeSingleton.class) {
                if (instance == null) { // 2nd check (under lock)
                    instance = new ThreadSafeSingleton();
                }
            }
        }
        return instance;
    }
}
```

- **Line-by-Line Breakdown & Complexity Analysis:** The `volatile` modifier is strictly required to prevent instruction reordering where a partially constructed object reference is published.

· 4. Practice Questions with Answers & Explanations

Practice Question 7.1

Question: Does `volatile` guarantee atomicity for `count++`?

Options:

- A) Yes
- B) No, because count++ consists of 3 distinct bytecode operations: read, modify, write
- C) Only on 64-bit JVMs
- D) Yes, if count is an integer

Answer: **B) No, because count++ consists of 3 distinct bytecode operations: read, modify, write** **Detailed Solution & Reasoning:** Use `AtomicInteger` with CAS (Compare-And-Swap) for atomic increments.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 7.1 (Amazon)

- **Interview Question:** _What is the difference between Callable and Runnable?_
- **Model High-Scoring Answer:** Runnable's `run()` method returns void and cannot throw checked exceptions. Callable's `call()` method returns a generic parameterized result (`Future<V>`) and can throw checked exceptions.
- **Interviewer Follow-Up & Pro Tip:** ? Mention pairing Callable with ExecutorService and Future.get().

CHAPTER 8

Java 8+ Functional Programming & Streams API

Key Principle / Exam Takeaway: Master Lambda expressions, Functional Interfaces (@FunctionalInterface), Streams API (map, filter, reduce), and Optional pattern.

Core Curriculum Topics Covered in this Chapter:

- Functional interfaces: Function, Predicate, Consumer, Supplier
- Lambda syntax and lexical scoping (effectively final variables)
- Streams API pipeline: Source -> Intermediate Operations (lazy) -> Terminal Operation (eager)
- Collectors: groupingBy, partitioningBy, joining, and toMap
- Optional<T> best practices for eliminating NullPointerException

· 1. Core Concepts & In-Depth Explanation

Java 8 introduced functional programming concepts to declarative data processing:

- **Functional Interface:** Interface with exactly one abstract method (SAM), annotated with `@FunctionalInterface`.
- **Lambda Expressions:** Anonymous functions passed as data. Can access local variables from enclosing scopes only if they are `final` or effectively final.
- **Streams API:** Computations on collections using lazy execution pipelines. Intermediate operations (e.g., `filter()`, `map()`, `sorted()`) do not execute until a Terminal Operation (e.g., `collect()`, `reduce()`, `count()`) is invoked.

· 2. Key Formulas, Rules & Complexity Classes

1. **Stream Pipeline Structure:** $$\text{Collection} \to \text{.stream()} \to \text{filter/map (Lazy Intermediate)} \to \text{.collect() (Eager Terminal)}$$

· 3. Step-by-Step Solved Examples

Example 8.1: Group Employees by Department and Calculate Average Salary

- **Difficulty / Target Companies:** `Easy | Amazon, TCS Digital`
- **Problem Statement:** Given a list of Employees, group them by department name and compute average salary per department.

Step-by-Step Walkthrough & Logic:

- Step 1: Convert list to stream.
- Step 2: Collect using `Collectors.groupingBy()` with downstream collector `Collectors.averagingDouble()`.

Complete Solved Code / Calculation:

```
import java.util.*;
import java.util.stream.Collectors;

class Emp {
    String dept;
    double sal;
```

```
    Emp(String d, double s) { dept = d; sal = s; }
    public String getDept() { return dept; }
    public double getSal() { return sal; }
}

public class StreamDemo {
    public static Map<String, Double> avgSalaryByDept(List<Emp> emps) {
        return emps.stream()
                   .collect(Collectors.groupingBy(
                       Emp::getDept,
                       Collectors.averagingDouble(Emp::getSal)
                   ));
    }
}
```

- **Line-by-Line Breakdown & Complexity Analysis:** Executes in a single pass over the collection with clean, declarative syntax.

· 4. Practice Questions with Answers & Explanations

Practice Question 8.1

Question: What happens if you invoke multiple terminal operations on the same Stream instance?

Options:

- A) It recalculates results
- B) Throws IllegalStateException: stream has already been operated upon or closed
- C) Returns empty stream
- D) Automatically resets

Answer: **B) Throws IllegalStateException: stream has already been operated upon or closed** **Detailed Solution & Reasoning:** Streams are single-use traversals that cannot be reused once consumed.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 8.1 (Amazon)

- **Interview Question:** _Why should Optional<T> not be used as method parameters or class fields?_

- **Model High-Scoring Answer:** Optional is designed strictly as a method return type to signal the possible absence of a value without returning null. Using Optional as method parameters forces callers to wrap values unnecessarily (`Optional.of(val)`) and does not prevent callers from passing `null` into the Optional parameter itself. Furthermore, `Optional` does not implement `Serializable`, which breaks serialization frameworks when used as class fields.

- **Interviewer Follow-Up & Pro Tip:** ? Cite Brian Goetz's design guidelines for Optional.

CHAPTER 9

I/O Streams, Serialization & Modern Networking

Key Principle / Exam Takeaway: Master Byte streams vs Character streams, Buffering, Java Serialization mechanics, serialVersionUID, and Non-blocking NIO.2.

Core Curriculum Topics Covered in this Chapter:

- Byte streams (InputStream/OutputStream) vs Character streams (Reader/Writer)
- Buffered streams and I/O performance optimization
- Java Serialization, transient keyword, and serialVersionUID validation
- NIO.2: Path, Files, Channels, and Buffers
- Socket programming and non-blocking I/O (NIO Selectors)

· 1. Core Concepts & In-Depth Explanation

Java I/O handles streaming data:

- **Byte Streams:** Process raw binary data (images, compiled classes, audio) 8 bits at a time.
- **Character Streams:** Automatically handle character encoding/decoding (UTF-8) 16 bits at a time.
- **Serialization:** Converting active in-memory object graphs into byte streams for disk persistence or network transmission (`implements Serializable`). Marking a field `transient` prevents it from being serialized.

· 2. Key Formulas, Rules & Complexity Classes

1. **serialVersionUID Invariant:** If `serialVersionUID` in serialized stream does not match class loader version: $$\implies \text{Throws InvalidClassException}$$

· 3. Step-by-Step Solved Examples

Example 9.1: High-Performance File Copying with NIO.2 Files.copy()

- **Difficulty / Target Companies:** `Easy | Infosys, TCS`
- **Problem Statement:** Copy a file safely utilizing modern NIO.2 APIs with replacement.

Step-by-Step Walkthrough & Logic:

- Step 1: Obtain `Path` instances via `Path.of()`.
- Step 2: Use `Files.copy()` with `StandardCopyOption.REPLACE_EXISTING`.

Complete Solved Code / Calculation:

```
import java.nio.file.*;
import java.io.IOException;

public class FileCopyDemo {
    public static void copyFile(String src, String dest) throws IOException {
        Path srcPath = Path.of(src);
        Path destPath = Path.of(dest);
        Files.copy(srcPath, destPath, StandardCopyOption.REPLACE_EXISTING);
```

```
    }
}
```

- **Line-by-Line Breakdown & Complexity Analysis:** Leverages operating-system-level zero-copy file transfer primitives where available.

· 4. Practice Questions with Answers & Explanations

Practice Question 9.1

Question: What does the transient keyword signify on an instance variable?

Options:

- A) It is thread-safe
- B) It is omitted from the object's serialized byte stream
- C) It cannot be modified
- D) It is stored in Metaspace

> **Answer:** **B) It is omitted from the object's serialized byte stream** **Detailed Solution & Reasoning:** Transient fields are skipped during default serialization and reset to default values upon deserialization.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 9.1 (Oracle)

- **Interview Question:** _Why should every Serializable class explicitly declare a private static final long serialVersionUID?_

- **Model High-Scoring Answer:** If a class does not explicitly declare `serialVersionUID`, the JVM runtime calculates a default hash based on class structure, fields, and method signatures. Even minor modifications (such as changing a method name or adding a private field) alter this computed hash, causing previously serialized objects to fail deserialization with an `InvalidClassException`. Declaring an explicit `serialVersionUID` guarantees version compatibility across compiler versions.

- **Interviewer Follow-Up & Pro Tip:** ? Explain that it maintains cross-version backwards compatibility.

CHAPTER 10

Java Placement Coding Patterns & Enterprise Architecture

Key Principle / Exam Takeaway: Master top enterprise design patterns (Factory, Builder, Strategy, Observer) and technical placement coding rounds.

Core Curriculum Topics Covered in this Chapter:

- Creational Patterns: Builder pattern with fluent API and Factory method
- Behavioral Patterns: Strategy and Observer patterns in Spring architectures
- Dependency Injection and Inversion of Control (IoC) fundamentals
- Placement coding drills: LRU Cache in Java, Custom ThreadPool, and Concurrent Queue
- Senior placement interview communication and code review best practices

· 1. Core Concepts & In-Depth Explanation

This capstone chapter covers enterprise architecture and structural design patterns tested in technical interviews:

- **Builder Pattern:** Separates complex object construction from its representation, providing fluent, readable object configuration without telescoping constructors.
- **Factory Pattern:** Defines an interface for object creation, delegating instantiation to specialized subclasses.
- **Strategy Pattern:** Defines a family of algorithms, encapsulating each one, making them interchangeable at runtime.
- **Dependency Injection (DI):** Objects receive their dependencies from an external assembler (such as the Spring IoC container) rather than instantiating dependencies internally via `new`.

· 2. Key Formulas, Rules & Complexity Classes

1. **Builder Pattern Fluent Structure:** $$\text{User } u = \text{User.builder}().\text{name}("Alice").\text{email}("a@test.com").\text{build}();$$

· 3. Step-by-Step Solved Examples

Example 10.1: Builder Pattern Implementation with Fluent API

- **Difficulty / Target Companies:** `Medium | Amazon, Google, Flipkart`
- **Problem Statement:** Implement the Builder pattern for a `HttpRequest` configuration object.

Step-by-Step Walkthrough & Logic:

- Step 1: Private constructor taking static inner `Builder` class.
- Step 2: Static `builder()` method returning new `Builder`.
- Step 3: Fluent setter methods on `Builder` returning `this`.
- Step 4: `build()` method returning validated `HttpRequest`.

Complete Solved Code / Calculation:

```
public class HttpRequest {
    private final String url;
    private final String method;
    private final int timeoutMs;

    private HttpRequest(Builder builder) {
        this.url = builder.url;
        this.method = builder.method;
        this.timeoutMs = builder.timeoutMs;
    }

    public static class Builder {
        private String url;
        private String method = "GET"; // Default
        private int timeoutMs = 5000;  // Default

        public Builder url(String url) { this.url = url; return this; }
        public Builder method(String method) { this.method = method; return this; }
        public Builder timeoutMs(int timeoutMs) { this.timeoutMs = timeoutMs; return this; }

        public HttpRequest build() {
            if (url == null) throw new IllegalStateException("URL is mandatory");
            return new HttpRequest(this);
        }
    }
}
```

- **Line-by-Line Breakdown & Complexity Analysis:** Provides immutable object construction with clean, readable parameter configuration.

· 4. Practice Questions with Answers & Explanations

Practice Question 10.1

Question: Which design pattern is utilized when passing a Comparator lambda to `list.sort()`?

Options:

- A) Singleton Pattern
- B) Strategy Pattern
- C) Decorator Pattern
- D) Prototype Pattern

Answer: **B) Strategy Pattern** **Detailed Solution & Reasoning:** The sorting algorithm remains fixed while the comparison strategy is supplied interchangeably at runtime.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 10.1 (Amazon)

- **Interview Question:** _What is Inversion of Control (IoC) and how does Dependency Injection realize it?_

- **Model High-Scoring Answer:** In traditional programming, high-level components control execution flow by directly instantiating their low-level dependencies using `new`. Inversion of Control inverts this control: an external container manages component lifecycles. Dependency Injection is the mechanism realizing IoC: instead of a class looking up or instantiating its dependencies, the dependencies are injected into the class via constructors, setters, or fields at runtime.

- **Interviewer Follow-Up & Pro Tip:** ? Connect IoC to decoupling and simplified unit testing.

· Campus Placement Certification & Final Exam Policy

To earn Placement-Ready Master status, students must complete all chapter practice questions, review all solved examples, and score at least 80% on the comprehensive subject assessment.